

Date: _/_/_

Memory Management Techniques

Contiguous

used in 1960's

- ↳ fixed / static partition
- ↳ variable / dynamic partition (allocate at run time)

* No spanning is allowed

Non Contiguous

- Paging
- Multi level
- Inverted
- Segmentation
- Segmented paging

Contiguous

- ↳ Continuous amount of Memory is allocated.
- State -> Mainframe computers.

(at time of system configuration)

↳ Fixed

• No. of partition are fixed

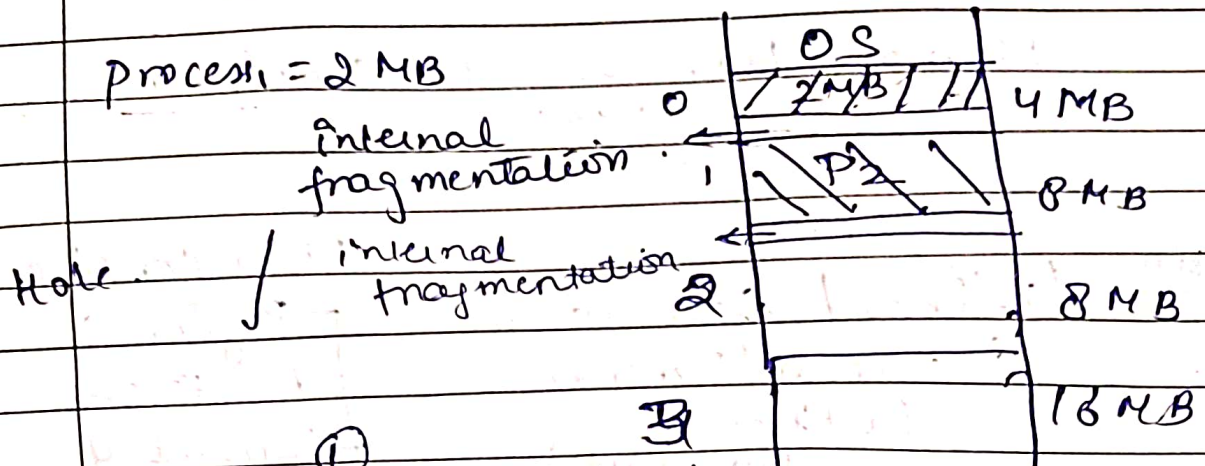
• Size of partition may or may not be same

• Contiguous allocation, so * spanning is not allowed

OS		OS
	4MB	8MB
	8MB	
		8MB
	8MB	
	4MB	

$P_2 = 7 \text{ MB}$

Date: __/__/__



Disadv. → ① Internal fragmentation

② Limit in process size
as 16 MB is limit here

③ as no. of partitions are fixed.
if we want 5 to 6 processes in
RAM.

i.e. Degree of multiprogramming
is limited

→ Removed using concept of
virtual programming.

④ External fragmentation

Adv.:

Date: _/_/_

1. Easy to Implement
2. prevent process from interfering with each other's memory space, improving security & stability of system.

Dynamic / Variable

*. Partition are Not made during before execution or during system configuration.

- Initially, RAM is empty. Partition are made during runtime acc. to process need instead of partitioning during system configuration.
- Partition size = size of incoming process.
so no internal fragmentation
- Partition size varies acc. to need of process so internal fragmentation can be avoided.
- Partition No. in RAM is not fixed depend on no. of incoming processes & M.M. size.

Date: __/__/__

OS	
P ₁ = 2 MB	Block size = 2 MB
P ₂ = 7 MB	Block size = 7 MB
P ₃ = 8 MB	Block size = 8 MB
Empty space of RAM	

Adv:

1. No Internal fragmentation
2. No restriction on degree of Multiprogramming
3. No limitation on process size

DisAdv:

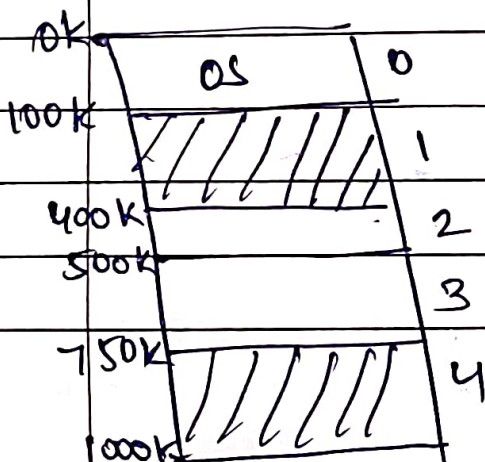
1. External fragmentation → as process must be continuously present in MM to get executed

2. ~~Difficult~~ Difficult to Implement

⇒ Once partition are defined, OS needs to keep track their status, such as free, allocation. Current partition status or attributes are collected in data structure called PDT: (Partition description table.)

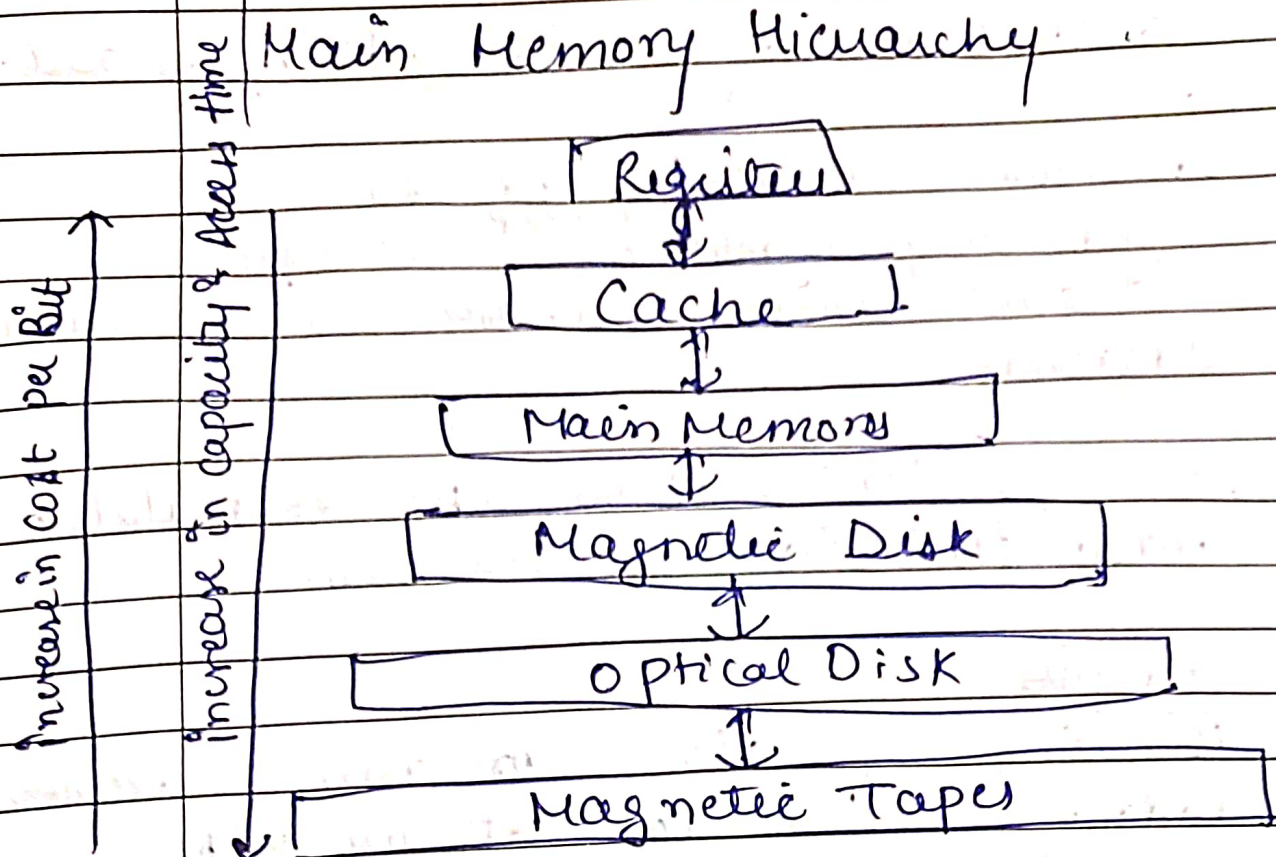
In PDT, partition are defined by its base address, size & status.

Cx,	Partition	Base	Size	Status
	0	0 K	100 K	allocated
	1	100 K	300 K	Free
	2	400 K	100 K	Allocated
	3	500 K	250 K	allocated
	4	750 K	250 K	free.



Compaction + Overcome External fragmentation.

— Move process in memory in such a way scattered pieces of unused memory (holes) can be placed together; so that other process demanding continuous memory for it can use.



Memory Mgmt functions →

1. Allocate & deallocate Memory before & After process Execution
2. To keep track of used memory space by processes.
3. Minimize fragmentation Issues
4. Proper utilization of M.M.
5. Maintain Data Integrity while process execution

Relocatable Add → are temporary add.

Address in source program are generally symbolic ^{ex: jump} compiler bind these addresses to relocatable addresses.

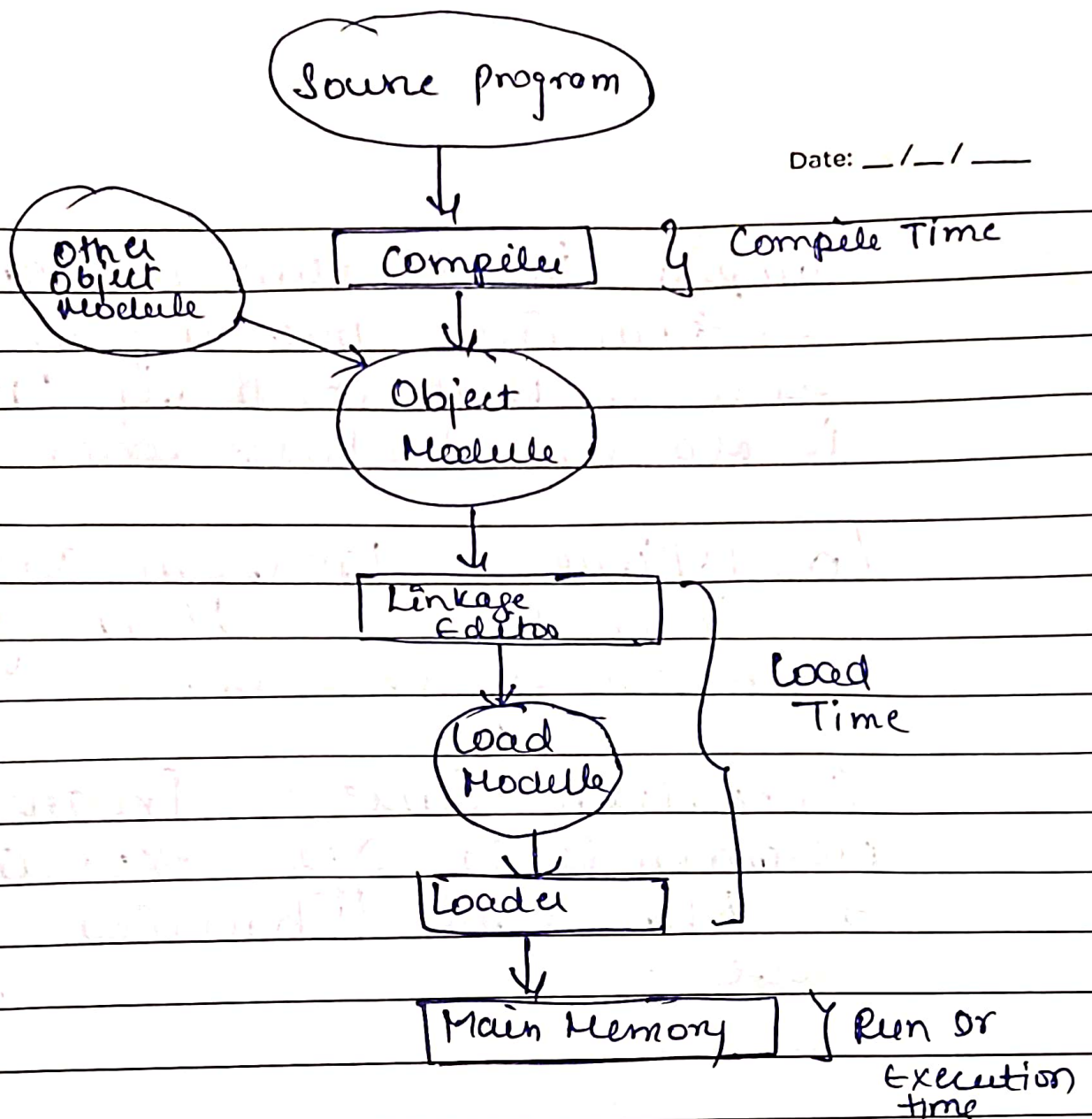
Loader & linker bind the relocatable Add. to absolute Address.

Compiler

⇒ Binding of data & program to Memory address can be done at any of the following step -

① Compile Time → Binding at compile time generate absolute Address, where a prior knowledge is required that where a process resides in Memory.

② Load Time → At compile time if it is not known that where a process will reside in memory then compiler must generate relocatable address.



Multi-step Processing of a user program.

final binding is delayed till load time

- ③ Execution Time → Allow process to move from one memory segment to another during ~~run~~ run time. Final Binding is delayed until Run time.

Date: __/__/__

Compile Time → Computer program is translated into a format that is understood by CPU.
i.e. Machine Readable code.

Load Time → Program are loaded into memory with use of loader.

Execution Time → Instruction in program code are executed.
→ Run time libraries are used.

Date: _/_/_

Static & Dynamic Loading

↳ Load entire program into fixed address.

Dynamic - In dynamic loading, a routine is not loaded until it is called.

NOTE: Compile time & load time address binding generates - identical logical & physical address.

Address Binding generated at execution time results in different logical & physical address.

Address Binding process of mapping program's logical or virtual address to corresponding physical or M.M. Address.

→ Performed by OS on behalf of the application that needs to access memory.

Logical Address

- Generated by CPU
- Set of all logical address generated by program - Logical Address Space

Physical Address ^{is address of MM} → Obtained from Memory Address register.

Physical Address space → Set of all physical address corresponding to these logical addresses.

Computer Memory Size

4 Bit - Nibble Byte

1 Bit - 0/1

8 Bit - 1 Byte

1024 Byte - 1 KB 2^{10} byte

1024 KB = 1 MB 2^{20} byte

1024 MB = 1 GB 2^{30} byte

1024 GB = 1 TB 2^{40} Byte

1024 TB = 1 PB 2^{50} Byte

Best fit, first and worst fit are the most common strategies to select free hole from available holes-

1. first fit:- Allocate first free partition / hole large enough which can accommodate process.
→ finishes after finding first suitable free partition.

2. Best fit:- Allocate smallest free partition
→ Search entire list of free partitions and consider smallest hole that is adequate
→ Try to find hole that is close actual process size

3. Worst fit:- Allocate largest available free portion
→ Reverse of Best fit.

Mem Spanning - no division of processes

Date: __/__/__

Non Contiguous Mem Allocation

OS

Allow spanning → Allow process division and allocate its diff. memory locations.
(Non adjacent)

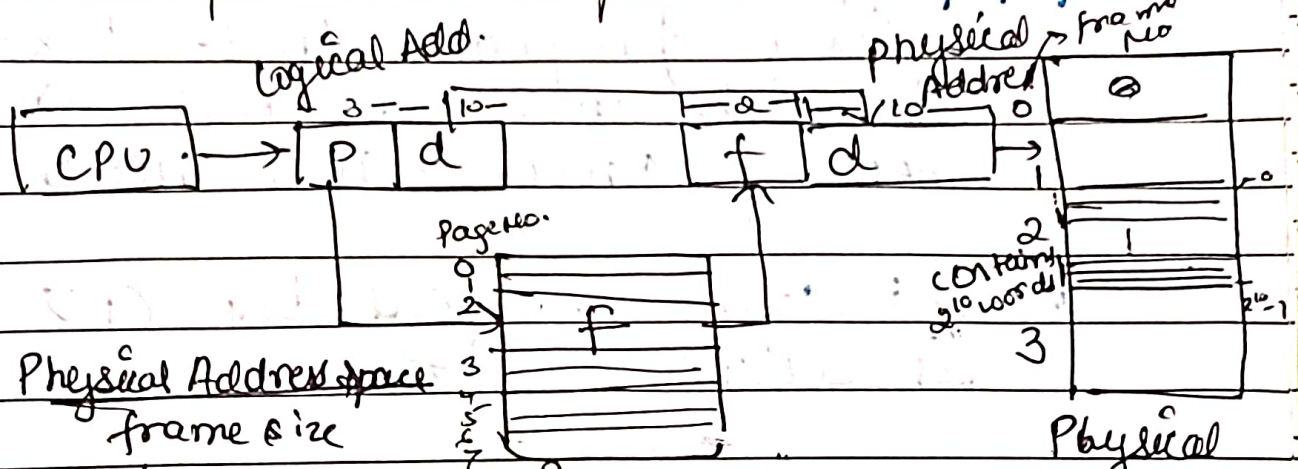
→ Entire process does not need to be stored in continuous block of memory.

→ use pointers to link non continuous memory blocks allocated to process
→ these pointers are used by OS to keep track of memory blocks allocated to processes & locate them during process execution

Date: _/ _/ _

Paging: In Secondary Memory, divide processes into pages - fixed size blocks. M.M. is divided into frames - fixed size blocks.

$$\text{frame size} = \frac{\text{process size}}{\text{page size}}$$



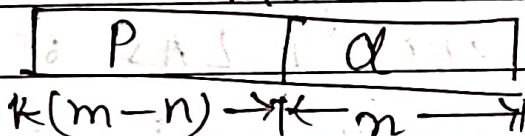
$$\text{No. of frames} = \frac{\text{Physical Address space}}{\text{frame size}}$$

$$= \frac{4K}{1K} = 4 = 2^2 \quad \text{Page table}$$

$$\text{No. of pages} = \frac{\text{Logical Address space}}{\text{page size}} = \frac{8K}{1K} = 2^3 = 8$$

Page No. | Offset page | frames | page offset

$$\text{page size} = \text{No. of pages} \times \text{Page table Entry in bits}$$



If capacity of P.M. is 'm' & size of each page is 'p' then No. of frames in P.M. will be

$$f = \frac{m}{p}$$

H/w Implemen

Paging provide better memory use + make it simpler to manage.

→ Support Virtual Memory Concept, allow program to be stored in disk and loaded into memory only when needed.

* Address Generated by CPU is divided into

Page No. → No. of bits required to represent page in Logical address space or page no.

Page Offset → No. of bits required to represent a particular word in page or page size. of LAS or word No. of page or page offset.

1. Physical Address is divided as
frame No. → No. of bits required to represent frame in PAs or frame no.

H/w Implementation of Paging

frame offset $\rightarrow$ No. of bits required to represent a particular word in frame

Paging H/w Implementation

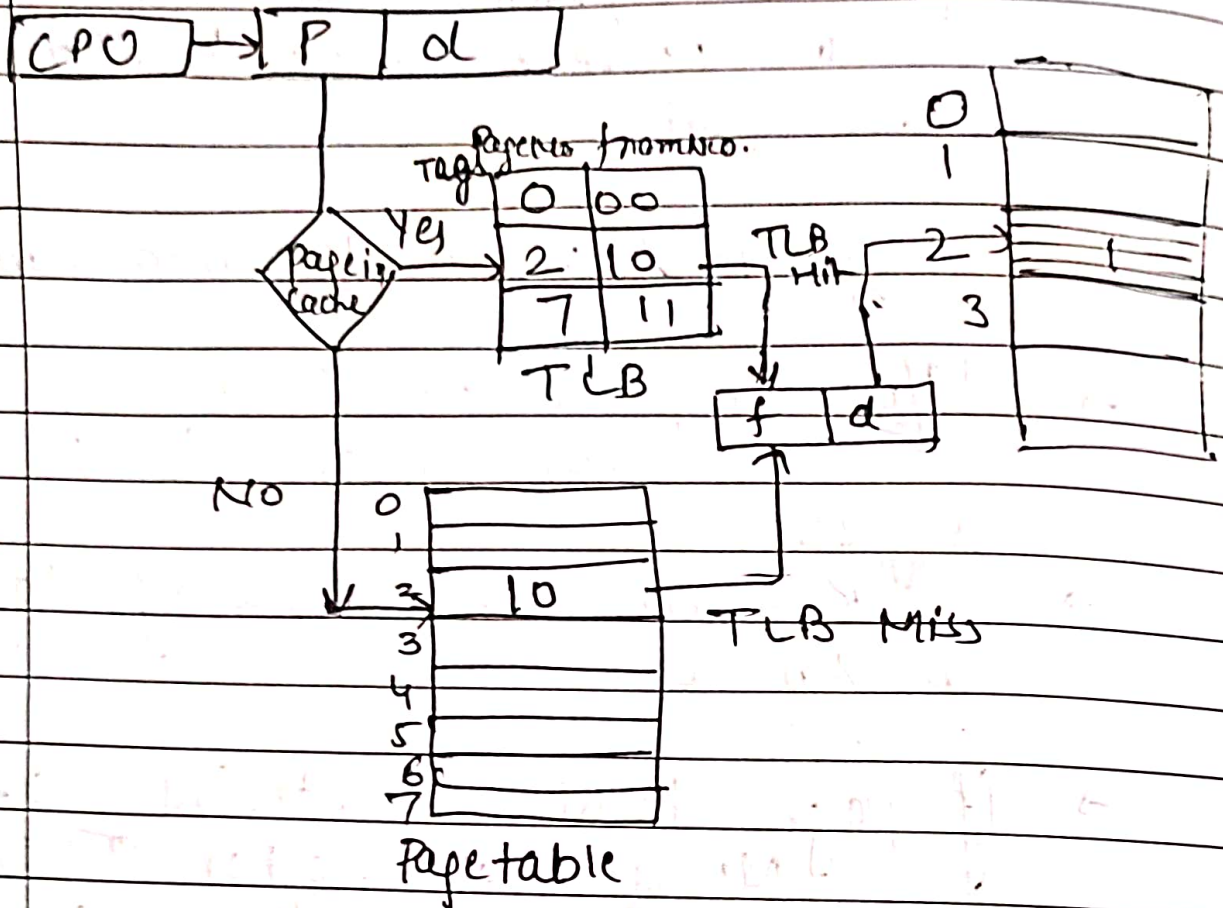
- $\rightarrow$ H/w Implementation of Page table can be done using dedicated registers.
- $\Rightarrow$ Registers are useful if Page table is small.
- $\rightarrow$ If page table contains large No. of entries then TLB, small, fast look up H/w cache is used.
- $\rightarrow$ TLB is associative, high speed memory
- $\rightarrow$ Each TLB entry consist of Tag & Value
- $\rightarrow$ When used, item is compared with all tags simultaneously. If found corresponding value is returned.

Eff. Hit - TLB Hit

page table is also in m.m.

Date: __/__/__

i.e. $TLB = \text{page No.} + \text{frame No.}$



Steps of TLB Hit

1. CPU Generates Virtual Addr
2. It is checked in TLB
3. Corresponding frame No. tells, where MM page lies.

TLB Miss :-

1. CPU Generates Virtual Add.
2. It is checked in TLB
3. Page No. is matched with page table
reside in M.M.
4. Corresponding frame No. is retrieved
which tells where MM page lies
5. TLB is updated with new PTE.

effective Memory Access Time.

$$= h(c+m) + (1-h)(c+nm)$$

h $\rightarrow$ hit ratio

m $\rightarrow$ memory access-time

c $\rightarrow$ TLB access-time

n $\rightarrow$ system level.

$n =$

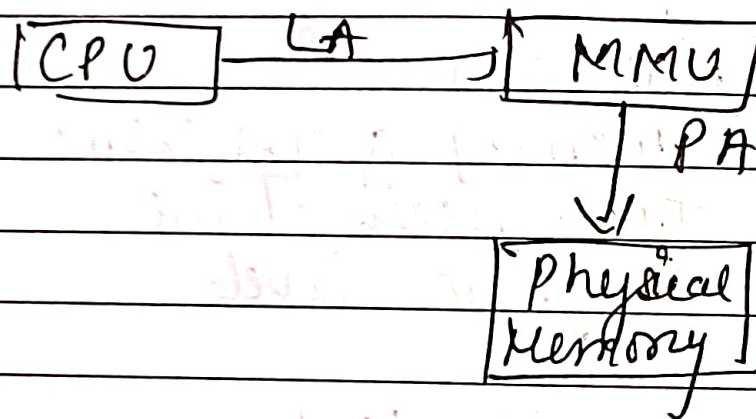
1	-	No	page table
2	$\rightarrow$	One	" "
3	$\rightarrow$	2	" "

Paging

- Eliminates External fragmentation
- Efficient Memory utilization
- Support VM.
- Easy swapping

MMU → technique used to convert logical address to physical address.

→ It is a h/w component



LSA $\rightarrow$ Represent size of process

Date: _/_/_

Given, LAS = 4GB PAS = 64MB
Page Size = 4KB

Compute No. of pages = ?

No. of frames =

No. of entries in Page table

Size of page table.

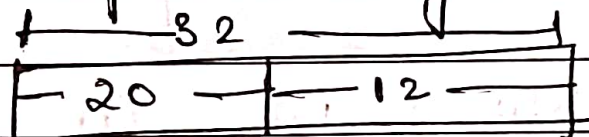
Memory is byte addressable

Soln: Logical Address = $\frac{4GB}{4KB} = \frac{2^{32}}{2^9}$ bits

i.e. Logical address = 2^{32} .

i.e. = 32 bits are used

to represent logical address



MSB

LSB

Page No.

offset / page size

page size = 4KB = $2^9 \cdot 2^{10} = 2^{12}$

i.e. 12 bits are used to represent page size / offset.

Page No. = $32 - 12 = 20$ bits

No. of pages = 2^{20}

Date: __/__/__

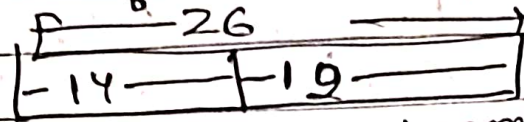
Physical Address Space

$$= 64 \text{ MB}$$

$$= 2^6 \cdot 2^{20}$$

$$= 2^{26}$$

PA is of 26 bits



frame no.

frame offset / frame size

$$26 - 12 = 14 \text{ bits}$$

$$\boxed{\text{No. of frames} = 2^{14}}$$

No. of entries in page table

= No. of pages in process

$$\Rightarrow \text{No. of entries} = 2^{20} \text{ i.e. } 1 \text{ M.}$$

$$\begin{aligned} \text{Size of Page table} &= \text{No. of entries in pg table} \times \text{frame bits} \\ &= 2^{20} \cdot 14 \text{ bits} \end{aligned}$$

Segmentation :-

Process is divided into segments, which are not of equal size.

~~Segment table~~ ~~it~~ ~~2D~~ ~~stores~~ ~~info~~
~~about~~ ~~all~~ ~~segments~~ ~~in~~

Segment table stores all information about all segments

→ It is responsible for mapping 2D logical address into 1D physical address.

Its each table entry has -

Base address → Contains starting physical address where segment resides in memory

Segment limit → also known as segment offset. Specifies length of segment.

Date: __/__/__

Seg. No.

Seg. No.	Base Address	Limit
0	500	600
1	2500	800
2	1500	400
3	4600	200
4	3800	400

Logical Address Space

Seg. Table.

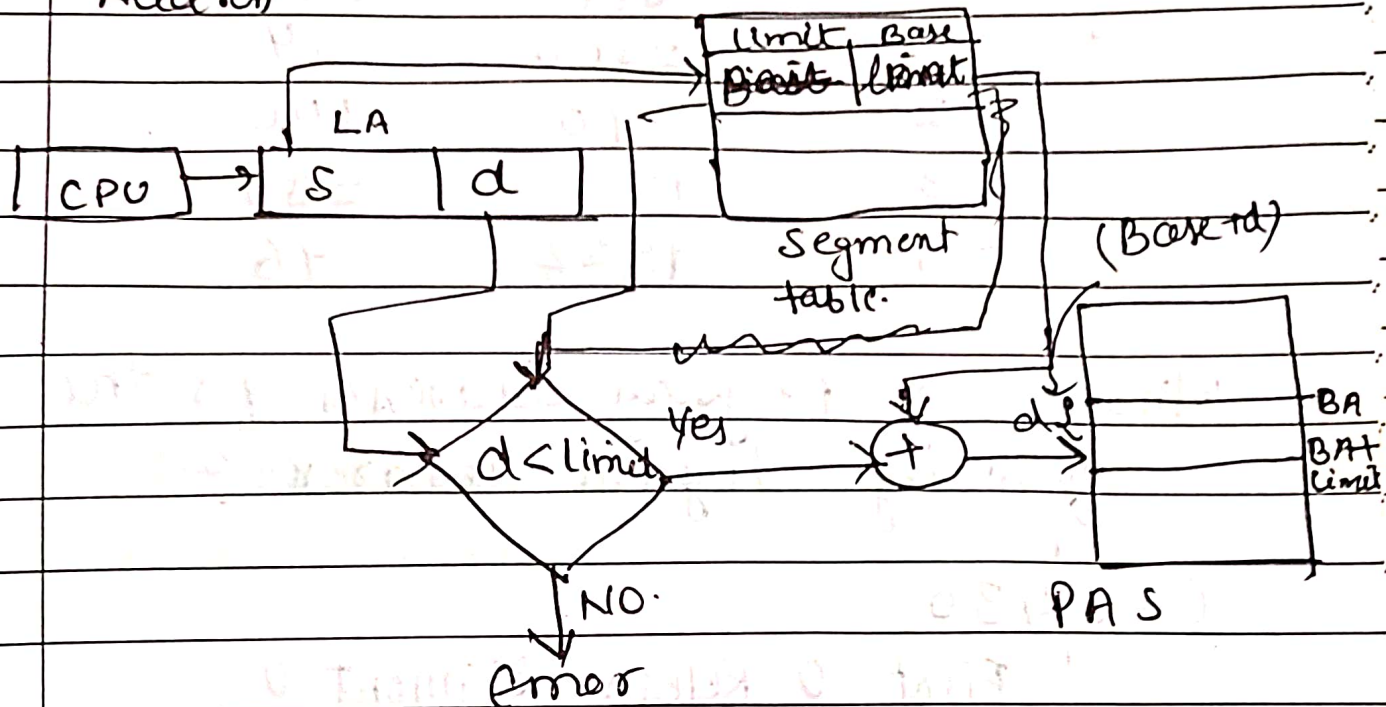
- Adv: 1. Reduced internal fragmentation
2. Segment table consumes less space compared to page table
3. Provide better CPU utilization
4. Segment size is decided by user but in paging H/w decides page size.

Disadv: 1. External fragmentation

2. Overhead in maintaining a segment table for each process.
- 3.

Physical Address Space

Translation of 20 L.A. to 1D physical Address



$$PA = \text{Offset} + \text{Base}$$

S → Segment No. → No. of bits required to represent segment.

d → segment offset → No. of bits required to represent position of data within a segment
→ 'inst' offset

Seg. Base → contain physical address where segment reside in memory

seg. limit → length of segment

Ques

Segment	Base	Length
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

What are physical addresses for the following logical address?

i.) 0/430

First 0 Refer Segment 0

430 $\rightarrow$ offset value

$$PA = 430 + 219 = 649$$

ii.) 1/10

Seg-1

$$PA = \text{offset} + BA$$

$$= 2300 + 10 = 2310$$

iii.) 2/500

$$PA = 90 + 500 = 590$$

Offset < limit conditⁿ false
illegal address

iv.) 3/400

$$PA = 1327 + 400 = 1727$$

v.) 4/112 $PA = 112 + 1952$ But 112 > 96 illegal

Date: __/__/__

Page Table Entries

		Optional field			
Frame No.	^{present} valid(1)/	Protection	Reference	Caching	Dirty
	invalid(0) absent	Rwx	(0/1)	↓ enable/ disable	
Mandatory					

Caching → whether page to be cached in Cache Memory

It is disabled when we need fresh data.

Reference Bit → useful for page replacement Algo

Dirty → write Back policy → Page that is modified should be on Hard disk

Protection (R, R/w, x) → whether read only or can be written.

Present/Absent → (Page fault)

Ques

Memory = 200 ns

ii.)

75% are found in TLB that means hit

$$\text{hit ratio} = 0.75 = 75\%$$

$$\text{Miss} = (1 - 0.75)$$

$$T_{\text{eff}} = 0.75 \times (0 + 200 \text{ ns}) + (1 - 0.75) (0 + 200 + 200)$$

Ques

A paging scheme using TCB,

TCB Acc. time 10 ns &

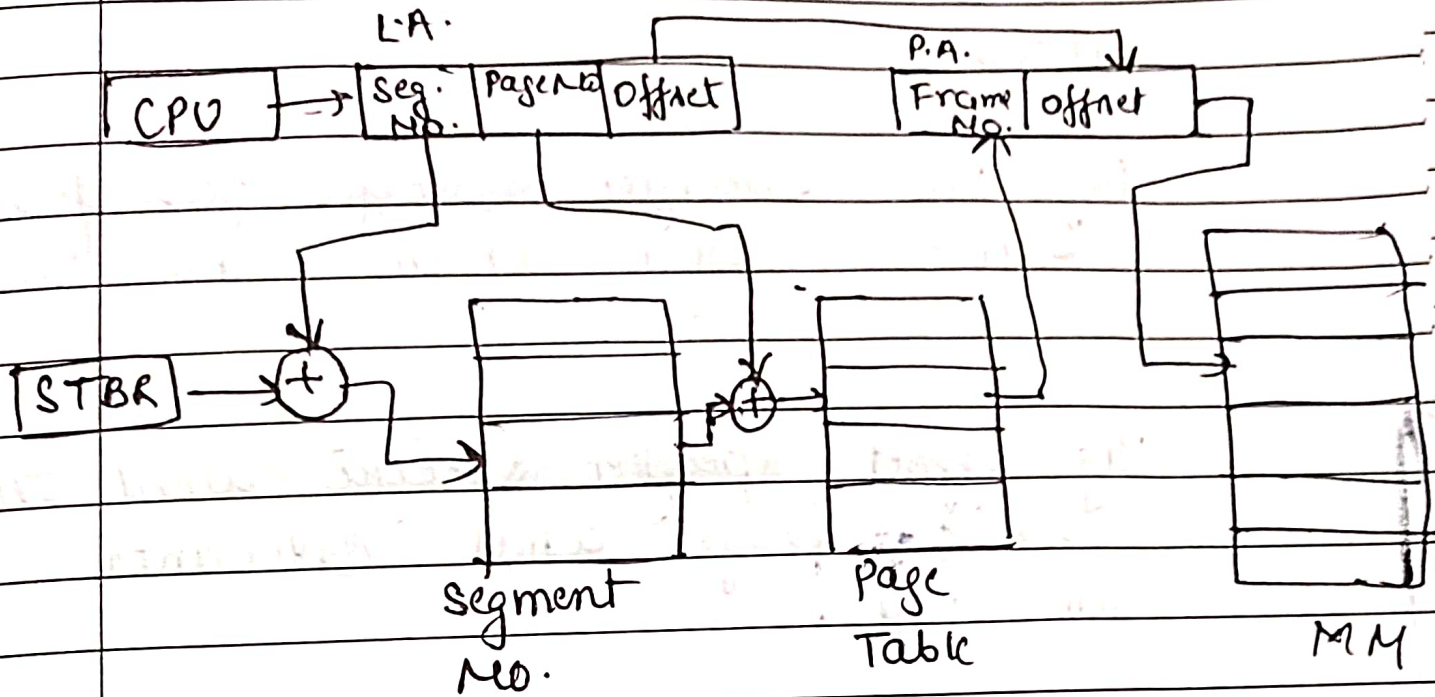
MM acc. time = 50 ns

Calculate effective memory access time in (ns).

TLB hit is 90% & No page fault.

$$\begin{aligned} \text{E. Memory AT} &= .9(10 + 50) + .1(50 + 50) \\ &= 65 \text{ ns.} \end{aligned}$$

Segmentation with Paging



- Implements Combination of paging & segmentation
- Process is divided into segments and then each segment is divided into pages.
- These pages are stored in frames in MM.
- A page table is exist for each segment that tracks frames storing pages of that segment

Seg. No specifies $\rightarrow$ specific segment that CPU wants to read the data.

Page No specifies specific page of that segment from which CPU wants to read data.

Page offset specifies specific word on that page that CPU wants to read.

Adv: 1. Reduce Memory usage

2. Page Table size is limited by segment size.

3. Resolves External fragmentation

4. Seg. Table contains exactly one entry corresponding to each segment.

Disadv:

1. Suffers from internal fragmentation
2. Complex.

V.M. (Virtual Memory) it is an illusion that computer system possesses more memory that it is actually having.

→ This illusion makes process independent of size of real memory.

→ It is a section of hard disk that's setup to emulate the computer's RAM.

→ VM is memory management capability of an OS that uses HW & SW to allow computer to compensate for physical memory shortage by temporarily transferring data from RAM to disk storage.

Can be implemented using Demand paging or segmentation

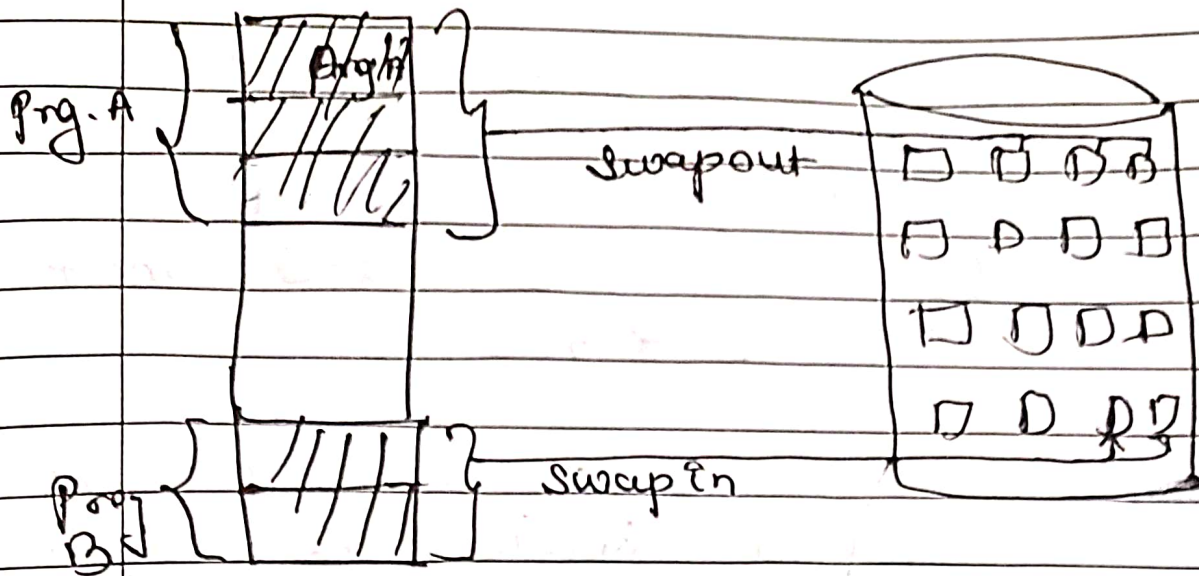
But swapin and swapout of segments are complex so paging is preferable.

Demand Paging

- Page is brought into memory for its execution only when it is demanded. Otherwise it remained in backing storage (disk).
- uses combination of paging & swapping
- Complete program is stored in disk in form of pages
- Swapping is used to swap content of program from disk to MM when it is required.

Adv.: Reduce Memory Requirement
swap time is reduced
increases deg. of multiprogramming

Date: __/__/__



M.M.

Disadv. $\rightarrow$ page fault $\rightarrow$ It is an interrupt that occurs in OS when a program attempts to access a page in memory that is not currently loaded into physical RAM.

Thus, running a program that is not entirely in memory would benefit both the **system** and the **user**.

8.1.3 Definition of Virtual Memory

Virtual memory is a memory hierarchy consisting of a computer system's memory and a disk, that enables a process to operate with only some portions of its address space in memory.

8.1.4 Implementation of Virtual Memory

Virtual memory is implemented by **demand paging**. It can also be implemented in a **segmentation** system. Several systems proved **paged segmentation** where segments are broken into pages but the operating system can implement this scheme with demand paging. **Demand segmentation** can also be used to provide virtual memory. The segment replacement algorithm are very complex in use because of **variable** sizes of segments while **page replacement algorithm** are easy to use and implement.

8.2 DEMAND PAGING

→ Paging
→ Swapping

In Demand paging, a page is brought into the memory for its execution only when it is demanded, otherwise it is remained in backing storage (disk). The name seems to have been derived from demand feeding – a policy used for feeding the baby by mothers in which the food is given to the baby only when he cries for it.

This method is the combination of **paging** and **swapping** method. The main requirement of this method is that the complete program should be present in the backing storage (disk) in the form of pages. Swapping is used to swap the contents of the program from disk to main memory when it is required.

In this method a "lazy swapper" is used, which swaps the pages when it is needed. Swapping requires removing the unused contents from main memory to disk and loading the required contents of program from disk to main memory (Figure 8.2).

Advantages:

- (i) Reduced memory requirement
- (ii) Swap time is also reduced
- (iii) Increase degree of multiprogramming.

lazy swapper → pager.
• we view a process as a sequence of pages, rather than one large continuous address space.
Swapper manipulates the whole process when a page is concerned with the individual pages in a process.

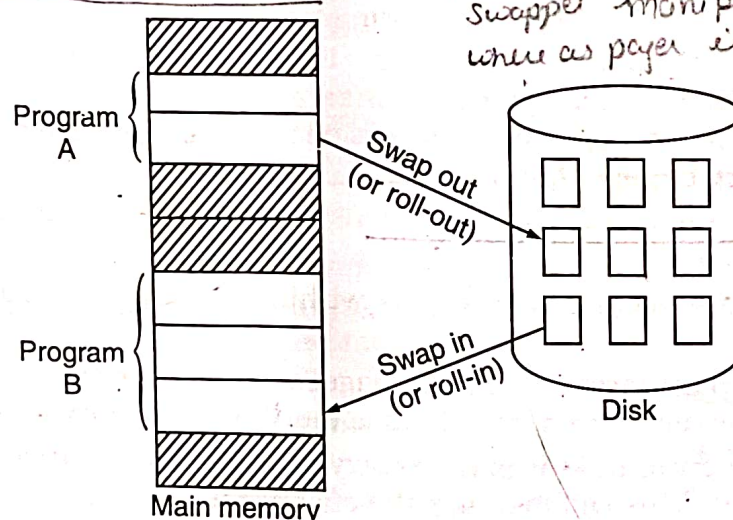


Fig. 8.2 Swapping with paging.

Disadvantages: *Page fault*

Using this method swapping is done using hit-and-trial method. For example, if a program is having 5 pages : 0, 1, 2, 3, 4 and out of these only pages 0, 3, 4 are loaded into memory at the time of execution state. If the program tries to access the address of page number 2, then there will be an error; as this page is not present in the main memory since it was not loaded at the time of swapping from disk to main memory. This error is called as **page fault error**.

This error is noticed by **paging hardware** while translating the address using the page table. The entire program is present in back storage in the form of pages (Figure 8.3).

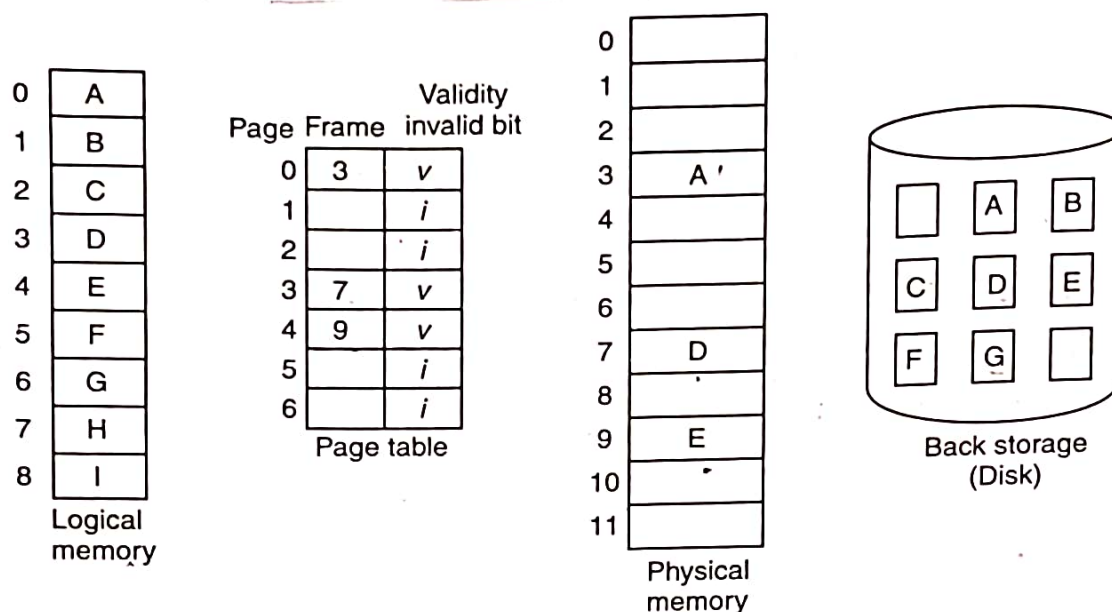


Fig. 8.3 Example of demand paging.

8.2.1 Performance Measurement

Page Fault

In many systems, when a process is executing with only a few pages of memory, and when an instruction is encountered which refers to any instruction or data in some other page which is outside the main memory (i.e., on the disk), a page fault will take place to occur. In this case, operating system must bring the required page into the memory before the execution of that instruction can restart. In this way, the number of pages in the physical memory for that process gradually increases with the page faults. After a while, when a sufficient number of pages are build up, the pages are normally found in the memory and then the frequency of page fault reduces.

The performance of demand paging is measured by computing the **effective access time** for demand paging system.

The general memory access time " m_a " is within the range of 500 nanoseconds to 2 microseconds.

When there is no page fault i.e., all the pages required for execution of a program is present in memory then effective access time is **similar** to memory access time.

If the page fault occurs, then it is necessary to bring the required page into memory from secondary storage. This will increase the effective access time.

Hence, the effective access time depends upon the probability of page fault 'p' and hence it is computed as :

$$E_a = [(p \times \text{"page fault time"}) + ((1 - p) \times m_a)] \quad \dots(8.1)$$

where **page fault time** is the time required to service the page which consists of :

- (i) Service the page fault interrupt
- (ii) Swap the page.
- (iii) Restart the process.

20 msec.
ma = 200ns

EXAMPLE 8.1 Suppose the average page fault service time is 20 milliseconds and memory access time is 200 nanoseconds. Suppose, we wish to have less than 10% degradation in memory access when a page fault occurs, what must be the page fault rate less than ?

Solution: Given,

$$\text{Page fault time} = 20 \times 10^{-3}$$

$$\text{and memory access time} = 200 \times 10^{-9} \text{ s}$$

$\therefore$ effective memory access time

$$< (200 \times 10\% + 200) \times 10^{-9} \text{ s}$$

$$< 220 \times 10^{-9} \text{ s.}$$

From equation (8.1),

$$E_a = [(p \times (20 \times 10^{-3}) + ((1 - p) \times (200 \times 10^{-9}))]$$

For the required result,

$$E_a < 220 \times 10^{-9} \text{ s.}$$

$$\text{i.e., } p \times (20 \times 10^{-3}) + (1 - p) \times (200 \times 10^{-9}) < 220 \times 10^{-9}$$

$$\text{or } -2 \times 10^{-7} p + 2 \times 10^{-2} p + 2 \times 10^{-7} < 220 \times 10^{-9}$$

$$\text{or } p(2 \times 10^{-2} - 2 \times 10^{-7}) < 220 \times 10^{-9} - 2 \times 10^{-7}$$

$$\text{or } p(10^{-2} - 10^{-7}) < 110 \times 10^{-9} - 10^{-7}$$

$$\text{or } p \left[\frac{1}{10^2} - \frac{1}{10^7} \right] < \left[\frac{110}{10^9} - \frac{1}{10^7} \right]$$

$$\text{or } p < .00001 \text{ (approximately)}$$

This means that the probability of occurring a page fault should be less than .00001 to achieve a degradation of less than 10%.

8.3 | PAGE REPLACEMENT ALGORITHMS

8.3.1 Principle of Page Replacement

The page replacement is done by **swapping** the required pages from backup storage to main memory and vice-versa. This swapping is done by checking the contents of physical memory. If there is a free frame in the memory then **swap-in** the required page into the frame which is free.

In case, if there is no free frame in physical memory then first find the frame which is not currently in use. The content of this frame is **swapped-out** from memory to backup storage. Then bring the required page in the frame which is now free.

✓ The changes are also made in **page table** to reflect that transfer of pages from main memory to backup storage and from backup storage to main memory.

Following steps are performed in page fault **routine** for page replacement :

- (i) Find the location of desired page on backup storage.
- (ii) Find the free frame:
 - (a) if the frame is free; use it.
 - (b) otherwise find the frame which is not currently being use. This is called as "VICTIM FRAME"
 - (c) write the contents of victim frame on back store and change the page table entries to indicate that the page is no longer in the main memory.
- (iii) Read the desired page into free frame; change page table and frame entries.
- (iv) Restart the user process.

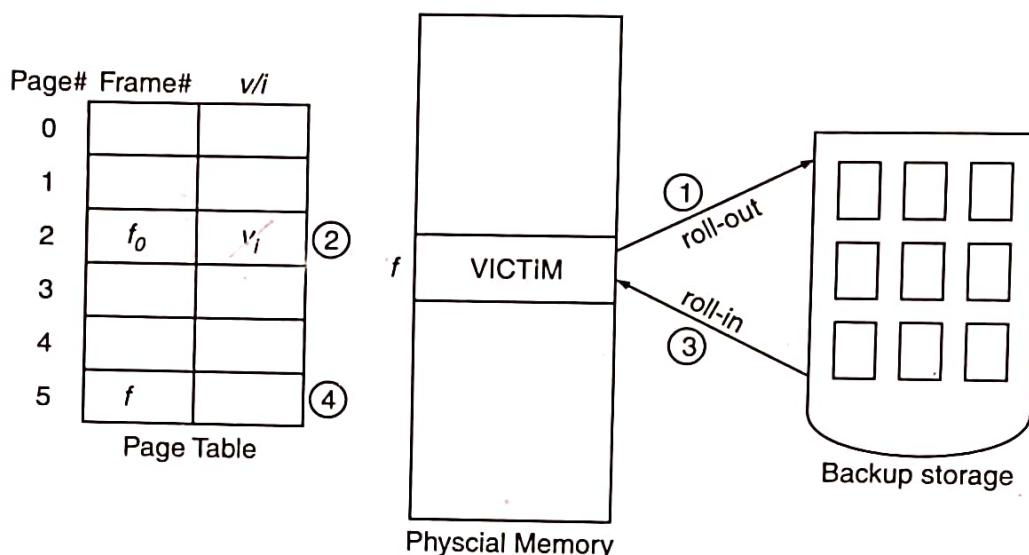


Fig. 8.4 Examples of page replacement.

- ① Swap out the Victim frame; ② Update its page table entry
- ③ Swap in the desired page ; ④ Update its page table entry.

8.3.2 Algorithms

Each operating system uses different page replacement algorithms. To select the particular algorithm, the algorithm with lowest page fault rate is considered. A page replacement algorithm is evaluated by running the particular algorithm on a string of memory reference and compute the page faults. The string of memory reference is called as **Reference String**. This string is generated by using either of the two methods :

- (i) By random number generator.
- (ii) Tracing the given system according to the address of each memory reference.

In the second case large amount of data is generated. This data will comprise of the address to which the references are made in some particular period of time. Hence, there is a need to reduce this data to some form so that it can be properly handled.

For this following method can be used :

- (i) For a given page size, only consider the page number and not the entire address to

which the reference has been made.

- (ii) If we have reference to page "p", then any immediate reference to that particular page will never cause the page fault.

EXAMPLE 8.2 Consider the following reference string generated in some stipulated period of time: 0100, 0432, 0101, 0612, 0122, 0125, 0801, 0750 and if the page size is 100, then instead of considering the entire address only the page numbers are considered. This reduces the amount of data for the benefits of operating system.

So new reference string will be : 1, 4, 1, 6, 1, 1, 8, 7 etc.

To find the number of page faults occurred for a particular reference string it is necessary to have the information about the number of **frames** available. The relation is, as the number of free frames available increases, the page fault will reduced, i.e., the two entries are **inversely** proportional to each other.

8.3.2.1 First-in-First-Out (FIFO) Replacement Algorithm

In the method, the oldest page present in the memory is selected for replacement. For this, the page at the **head** of the queue is replaced and when the page is brought into the main memory then it is placed at the end i.e., **tail** of the queue.

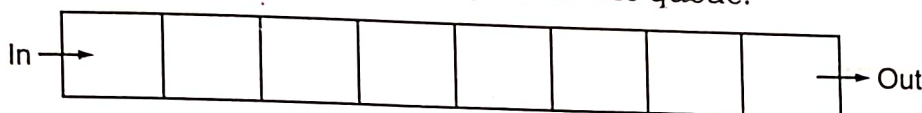


Fig. 8.5 Queue used for replacement algorithm.

• Easy to understand & program

EXAMPLE 8.3 If the contents of reference using is : 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3 and there are three frames available in the memory then the above algorithm works as :

P	P	P	P	P	P	P	P	P	P	P	P
7	7	7	0	0	1	0	4	2	3	0	0
	0	0	1	1	2	2	0	4	2	3	3
		1	2	2	3	3	3	0	4	2	2
7	0	1	2	0	3	0	4	2	3	0	3

Fig. 8.6 Implementation of FIFO algorithms.

Solution: Initially, assume that there is no page present in memory. After this, 7, 0, 1 enters, so three page faults will occur for each entry in memory. Next page 2 enters in memory but all frames are located, i.e., no frame is free. So, the oldest i.e., page 7 will be replaced and new entry of page 2 is made resulting fourth page fault. After this page 0 comes from reference string, but since it is already present in memory, so no page fault will occur. After this, page 3 comes; again a page fault will occur. Following these steps, we end total 10 number of **page faults** in this example.

8.3.2.2 Optimal Replacement Algorithm

• lowest page fault rate never suffer from Belady's anomaly

This algorithm has lowest page fault rate for all algorithms of page replacement. This algorithm states that: Replace that page which will not be used for longest period of time i.e., future knowledge of reference string is required. To understand this algorithm following example is considered.

EXAMPLE 8.4 Consider example 8.5 of given reference string and three available frames.

Apply optimal page replacement algorithm.

Solution: Firstly three pages: 7, 0 and 1 are entered in memory; assuming initially memory is empty; resulting three page faults. Now, no frame is free so page replacement will take place to enter the new page in memory.

By using optimal replacement algorithm, 7 can be replaced as in future it is not required also 1 is not required. So any of the two 1 or 7 can be replaced. But here we replace page 7, as it is older than 1. So, upto now, 4 page faults are occurred.

Next we have page 0 from reference string but since it is already placed in memory frames so No page fault will occur. Next page 3 is entered then we replace 1 as it is not required in future resulting 5 page faults. After this again 0 arrives so no page fault. Next 4 arrives now we have to replace 0 as it is required later then 2 and 3 in future. This results 6 page faults. Next pages 2 and 3 arrives but no page fault occur. After this 0 arrives then we have to replace either 2 or 4, but we replace 4 because of the same reason, resulting 7th case of page fault. Next, 3 arrive, so no page faults. So, overall we obtain **7 cases of page faults** in total.

7	0	1	2	0	3	0	4	2	3	0	3
		1	2	2	3	3	4	4	4	0	0
	0	0	1	1	2	2	3	3	3	3	3
7	7	7	0	0	0	0	2	2	2	2	2
P	P	P	P		P		P			P	

Fig. 8.7 Implementation of optimal algorithm.

8.3.2.3 Least Recently Used (LRU) Algorithm

In this algorithm, the page that has been not used for longest period of time is selected for replacement.

To understand this concept, considering following example :

EXAMPLE 8.5 Consider example 8.3 of given reference string and number of frames. Implement LRU algorithm.

7	0	1	2	0	3	0	4	2	3	0	3
7	0	1	2	2	3	3	4	4	4	0	0
	7	0	1	1	2	2	3	3	3	3	3
		7	0	0	0	0	2	2	2	2	2
P	P	P	P		P		P			P	

Fig. 8.8 Implementation of LRU algorithms.

Solution: Initially, it is assumed, that all frames of memory are free. Then pages 7, 0, 1 enters respectively resulting three page faults. Next page 2 will enter but it is not located unless a page is replaced from the memory.

Using LRU algorithm, page 7 will be replaced, as pages 0 and 1 are least recently used w.r.t. page 7 resulting 4 page faults. Next page 0 comes which is present in memory so no page fault. Next page 3 comes, now page 1 will be replaced as 2 and 0 are least recently used w.r.t. page 1. This will continue until all the pages of reference string is executed.

As a result in last, we obtain with total **7 number of cases of page fault**.

8.4 | THRASHING

The phenomenon of moving pages from primary to secondary storage or vice-versa consumes a lot of computer's energy but accomplishes very little useful results. This situation is called **thrashing**. A process is in thrashing, if it is spending **more** time in paging instead of their execution.

8.4.1 Cause of Thrashing

As we know that the operating system monitors CPU utilization. If CPU utilization is too low, we increase the degree of multiprogramming by introducing a new process into the system. A global page replacement algorithm is used that replaces pages with no regard to the process to which they belong. Now, suppose a process requires more frames than it starts faulting and taking pages away from other processes. These processes need these pages and so they also result in page faults, taking frames from other processes. These faulting processes must use the paging device to swap pages in and out. As they queue up for the paging device; the ready queue comes in empty state. As processes wait for the paging device, so CPU utilization decreases.

The CPU scheduler sees the decreasing CPU utilization and increases the degree of multiprogramming as a result. The new process tries to get started by taking frames from running processes, causing more page faults, and a longer queue for the paging device. As a result CPU utilization again degrades. Seeing this again CPU scheduler increases the degree of multiprogramming. So now the situation comes where **thrashing** will take place and CPU utilization is dropped. The page fault rate is increased tremendously. So no work is getting done, because processes are spending most of their time in paging.

This phenomenon is illustrated in **figure 8.10** in which CPU utilization is plotted against degree of multiprogramming. As degree of multiprogramming increases, CPU utilization goes on increasing although more slowly, until a maximum is reached. After this point of degree of multiprogramming is increased then **thrashing** is occurred and CPU utilization drops sharply after this point.

At this point, to **increase** CPU utilization and **stop** thrashing degree of multiprogramming should be reduced.

8.4.2 How the Effects of Thrashing can be Limited?

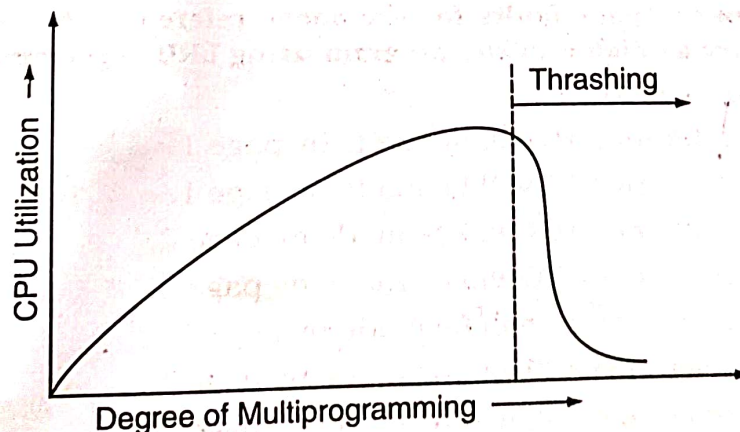


Fig. 8.9 Thrashing

✓ We can limit the effects of thrashing by using a local replacement algorithm (or priority replacement algorithm). With this algorithm, when one process starts thrashing it cannot steal frames from another process and cause the latter to also thrash. When more than one processes are in thrashing state then they will be in the queue for the paging device. The average service time for a page fault will increase, due to longer average queue of paging device. Thus, the effective access time will increase even for a process that is **not** thrashing.

8.4.3 What are Measures taken to Avoid Thrashing?

To prevent from thrashing a strategy called as page-fault frequency (PFF) is used.

Since, thrashing has a high rate of page fault. So, we can control the page-fault rate. When it is too high, we know that the process needs more frames. Similarly if the page fault rate is too low, then the process has too many frames. We can establish upper and lower bounds on the desired page fault rate (Figure 8.10). If the actual page fault rate exceeds the upper limit, we allocate that process to another frame. If the page fault rate falls below the lower limit, we remove a frame from that process. Thus, we can directly measure and control the page fault rate to prevent thrashing.

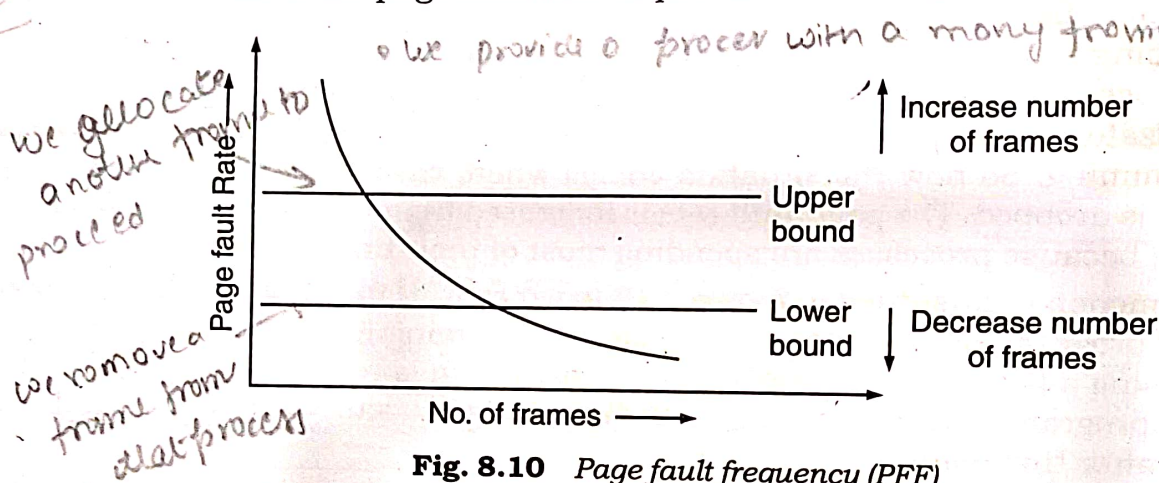


Fig. 8.10 Page fault frequency (PFF)

EXAMPLE 8.6 Consider the following sequence of memory reference from 460 words programs: 10, 11, 104, 170, 73, 309, 185, 245, 246, 434, 458, 364.

(i) Give the reference string assuming a page size of 100 words.

(ii) Find the number of page faults for the above reference string assuming 200 words primary memory available to the program using LRU replacement algorithm.

Solution: (i) Page size = 100 words.

- Memory reference 10 will be made by page 1.
- Memory reference 11 will be made by page 1.
- Memory reference 104 will be made by page 2.
- Memory reference 170 will be made by page 2.
- Memory reference 73 will be made by page 1.
- Memory reference 309 will be made by page 4.
- Memory reference 185 will be made by page 2.
- Memory reference 245 will be made by page 3.
- Memory reference 246 will be made by page 3.
- Memory reference 434 will be made by page 5.